Midterm Report - Optimizers

Group 1 - HawkAI Detection Team
Kevin Joseff Cabrera
Mohd Mujtaba Saighani
Thiago Segantini Nogueira
Sayamon Sittiprom
Yao-Fu Yang

AIGC-5000-0NA: Advanced Deep Learning
Prepared for: Hossein Pourmodheji

1. Project Introduction and Motivation

Neural networks are complex architectures with many customizable parameters to tune to optimize model performance. In past studies, we have explored many techniques and parameter tuning such as input standardization, weight initialization, batch normalization, dropout layers, number of hidden layers, hidden dimensions, early stopping, adjusting Epoch size, activation functions, and learning rate.

In this study, we focused on another customizable component, the optimizers. Optimizers are responsible for weight updates during the model training process. Given the complexities and nuances of different networks, there exists a variety of optimizers and their respective hyperparameters that aim to address different use cases when updating weights in a model. Therefore, this study aimed to research and compare the Adam, AdamW, and RMSProp optimizers in a controlled experiment to analyze their differences empirically.

2. Dataset Description

2.1 Dataset

This study used the KMNIST (Kuzushiji-MNIST) dataset, which contains 28x28 grayscale images containing ten classes of hand-written Japanese Hiragana characters. This dataset is available via PyTorch and comes with pre-made training and testing sets of 60,000 and 10,000 labeled images, respectively. Of note, the selected Japanese characters themselves have overlapping strokes and similarities, increasing the difficulty for classification.

Hiragana Characters:
お (o) | き (ki) | す (su) | つ (tsu) | な (na) | は (ha) | ま (ma) | や (ya) | れ (re) | を (wo)

3. Model Description

3.1 Model Requirements

In order to isolate any performance differences to the optimizers and their respective parameters, this study enforced an architecture containing an input layer of 784 neurons (28 x 28 flattened), first hidden layer of 128 neurons + ReLU activation, second hidden layer of 64 neurons + ReLU activation, output layer of 10 neurons + Softmax activation, and using cross-entropy loss for the loss function. In addition, we incorporated input standardization via mean/standard deviation, weight initialization using He initialization for the hidden layers and Glorot for the output layer, and batch normalization for each layer.

4. Optimizer Descriptions and Theoretical Background

4.1 RMSprop (Root Mean Square Propagation)

Overview: RMSprop was introduced to address the shortcomings of standard Stochastic Gradient Descent (SGD) and AdaGrad when training deep neural networks. It is an adaptive learning rate optimization algorithm that adjusts the learning rate for each parameter based on a model's gradient history. It's popularly known in the machine learning community from being introduced by Jeffrey Hinton during one of his lectures and not via a published, formal, peer-reviewed academic paper. This algorithm predates the widely used Adam optimizer from which the Adam was based on.

Mechanism: RMSprop maintains a moving average of squared gradients and uses that to dynamically adapt the base learning rate (L) for the current weight update. It achieves this by first calculating the gradient for the current step (G), updating the moving average of squared gradients (S) while factoring in the hyperparameter **decay rate (Beta)**, and then adjusts the learning rate by dividing it by the moving average + epsilon.

Numerical Stability: The epsilon parameter exists to prevent division by zero, which is particularly critical when the model enters flat gradient regions where the moving average of squared gradients might drop to zero.

Trade-offs: RMSprop is highly effective at navigating irregular, sparse, or volatile loss terrains. However, it can be slower than other methods on smooth gradients and may be prone to overfitting because it aggressively probes the loss surface, sometimes learning noise or outliers.

Parameters
1. ***Learning Rate (lr)***: The Learning Rate sets the baseline step size for updating the model's weights during training. Higher values create larger adjustments for faster training, while lower values make smaller, more precise updates.
2. ***Decay Factor (alpha)***: This factor controls the optimizer's memory window by deciding how much weight to give to recent gradient volatility versus historical data. A higher value (e.g., 0.99) considers a long history for stable, deliberate updates, whereas a lower value (e.g., 0.1) creates a highly reactive, aggressive step adjustment based only on immediate slopes.
3. ***Epsilon (eps)***: Epsilon is an ultra-small constant added to the denominator to prevent division by zero when gradients become extremely small or vanish. It ensures numerical stability, giving the algorithm the "tiny bump" it needs to keep learning without crashing the training loop.
4. ***Weight Decay (weight_decay)***: Weight decay applies an L2 regularization penalty to the loss function to keep model weights small. This discourages complexity and effectively prevents the neural network from overfitting to training data.
5. ***Momentum (momentum)***: This parameter accelerates gradient descent by accumulating a moving average of past directional steps. It helps the optimizer damp oscillations and reliably push through local minima or flat regions.

6. ***Centered (False)***: Enabling this boolean normalizes the gradients by an estimation of their variance rather than their uncentered second moment. This requires slightly more memory and computation but can lead to more stable and balanced training updates.

4.2 Adam (Adaptive Moment Estimation)

Adam (Kingma & Ba, 2015) is a widely used optimizer that integrates the concepts of **momentum** and **RMSprop**. It maintains two running averages for each parameter:

- **First moment:** The mean of gradients (acting as momentum).
- **Second moment:** The uncentered variance of gradients (acting as RMSprop).

By combining these, Adam computes adaptive learning rates for individual parameters, making it highly effective for problems involving sparse gradients or noisy data. A key feature of Adam is its inclusion of bias correction during early iterations, which prevents the initial moment estimates from being biased toward zero. While Adam is generally robust to learning rate selection, it is limited by its approach to L2 regularization; weight decay is applied in a "coupled" manner, which can distort the adaptive scaling process and potentially lead to poorer generalization.

Parameters

1. ***Learning Rate (lr)***: the global learning rate (step size)
2. ***β1 (betas=(β1, β2))***: decay rate for the first moment (momentum). Controls how much past gradients influence the current update.
3. ***β2 (betas=(β1, β2))***: decay rate for the second moment (RMSProp). Controls how much past squared gradients influence the scaling.
4. ***Epsilon (ε)***: small constant to prevent division by zero.
5. ***Weight Decay (weight_decay)***: applies L2 Regularization but in a coupled way. The penalty is applied to the gradients before the learning rate scaling adjustments.
6. ***De-coupled weight decay (decoupled_weight_decay)***: Simulates AdamW by applying L2 Regularization in a de-coupled way. Penalty is applied after scaling adjustments.

4.3 AdamW (Adam with Decoupled Weight Decay)

AdamW is an extension of the Adam optimizer so it also maintains the moving average of gradients and the moving average of squared gradients. However, it was created to address a fundamental limitation in the standard Adam optimizer regarding how L2 regularization is implemented. In the original Adam, the weight decay term is added directly to the gradient before the adaptive scaling is applied, meaning the decay is unintentionally distorted by the learning rate adjustments. In AdamW, the weight decay penalty is applied separately from the adaptive scaling adjustments, decoupling it from the additional processing step. Ultimately, this decoupling ensures that regularization behaves predictably and consistently across all parameters. As a result, AdamW typically achieves better generalization than standard Adam, as it avoids the uneven regularization effects caused by Adam's adaptive scaling.

Parameters

1. ***Learning Rate (lr)***: the global learning rate (step size)
2. ***Weight Decay (weight_decay)***: applies L2 Regularization but in a de-coupled way.
3. ***β1 (betas=(β1, β2))***: decay rate for the first moment (momentum). Controls how much past gradients influence the current update.
4. ***β2 (betas=(β1, β2))***: decay rate for the second moment (RMSProp). Controls how much past squared gradients influence the scaling.
5. ***Epsilon (ε)***: small constant to prevent division by zero.
6. ***Weight Decay (weight_decay)***: applies L2 Regularization but in a coupled way. The penalty is applied to the gradients before the learning rate scaling adjustments.
7. ***Maximize (maximize)***: Maximizes (maximize=True) the objective with respect to the params, instead of minimizing (maximize=False). In deep learning, we are minimizing the loss function not maximizing it.
8. ***Differentiable (differentiable)***: Whether autograd should occur through the optimizer step in training. Otherwise the step() function runs in a torch.no_grad() context. Setting to True can impair performance, so leaving it False doesn't run autograd through this instance.
9. ***Capturable (capturable)***: Whether this instance is safe to capture in a graph. Tensors are only capturable when on supported accelerators.

5. Methodology

5.1 Hyperparameter Strategy

After careful consideration and assessing the intensive compute necessary to run trials, our team proceeded with a manual systemic approach for all three optimizers. Our team members first conducted thorough research into each optimizer to learn about their hyperparameters potential effect on model training. Parameters were selected based on projected degree of impact and thresholds were chosen in relation to the baseline values (e.g., a discrete step of 0.1 increase/decrease or a tenfold increase/increase). To isolate the effects of each hyperparameter, the default values were selected as the first configuration, while subsequent trials adjusted one single parameter value while holding the others constant. After doing this initial comparison, then team members experimented with combination changes across parameters to diversify the configurations.

In addition, we leveraged Optuna to assess potential configurations for RMSprop. At a high level, this involved defining a specific range of values for each target parameter and specifying the number of trials to run through Optuna. This essentially applied random sampling of the continuous values between the defined ranges and evaluated them using the same k-fold cross-validation strategy as described below. Like the manual approach, the range values were selected based on discrete steps or a scalar deviation from default values. Due to compute intensity, we did not proceed with a grid search approach.

We used the validation accuracy as the assessment metric for both approaches, and the best configuration amongst all optimizers (with a separate slot allocated for RMSprop optuna) were selected for final model training and evaluation.

5.2 Cross-Validation Protocol

To isolate performance differences to hyperparameter configurations, we implemented a reusable 5-fold cross-validation strategy that can handle any optimizer with any configuration of its hyperparameters. We first defined the non-overlapping 5-fold splits using sklearn's Kfold class while using a fixed random seed; this design iteratively designates four folds as the training set and the fifth fold as the validation set until each fold has been used as the validation set exactly once. Then, for each permutation under test, we refreshed the torch manual seed with the same value to ensure continued reproducibility and consistency with initial states. Fresh model and optimizer instances were instantiated in each iteration to ensure complete independence from any previous states. The training dataset (KMNIST) was then partitioned into five non-overlapping sets according to the indices derived from the Kfold class. Then, the model ran 20 Epochs using the training data under training mode and the validation data under evaluation mode and metrics were captured for further evaluation and visualization.

5.3 Training Setup

Single Epoch
We first defined the functionality for a single epoch that could handle the training mode and the evaluation mode for our model. In training mode, this function completes the forward pass of our model to produce predictive outputs, calculates the loss and accuracy, calculates the gradients, and updates the weights via the designated optimizer. In evaluation mode, the function only completes the model forward pass and metrics calculations but does not calculate gradients or update weights. All metrics are then returned and stored for later evaluation/visualization.

Run Optimizer Configuration Trials
We then defined a reusable function that accepts the optimizer name, model type, and list of configurations to conduct model training. Each unique configuration is evaluated through the cross-validation function defined earlier and the metrics are captured from each configuration and stored for later evaluation and visualization. Most notably, this function identifies the best performing configuration and stores it for the final model evaluation.

Final Model Train and Evaluation
We next defined a final model training and evaluation function that is again optimizer-agnostic. After identifying the best configurations, each optimizer undergoes final model training using the full test set (no k-fold cross-validation) and the performance will be evaluated with the withheld KMNIST test dataset. Like the cross-validation protocol, the steps include setting the manual seed for reproducibility of initial state, instantiating a new model and optimizer, preparing the data loader objects from the original trainset and test set, followed by running through 20 epochs of training and evaluation. Metrics are again captured.

Synthesis and Visualizations
After defining all the functions above, we then selected at least four unique configurations for each optimizer for testing. We then fed the optimizer and configurations through the initial optimizer trials function to run cross-validation and identify the best configuration, then fed the optimizer and its best configuration through the final model training and evaluation. Finally, we generated plots of the training losses/accuracies and validation/testing losses/accuracies for all

optimizers under test. Final comparison plots were also used to visualize their accuracy differences and variance during cross-validation testing.

5.4 Final Metrics Summary

==========

FINAL METRIC REPORT: Adam

==========

1. Training Accuracy (Final): 95.27%
2. Cross-Validation: Mean: 0.9560; Std Dev: 0.0016
3. Test Accuracy: 89.69%
4. Final Losses:
   - Final Training Loss: 0.1461
   - Final Test Loss: 0.3702
5. Convergence Speed: 1 epochs to reach target threshold
6. Total Training Time: 247.12 seconds

==========

==========

FINAL METRIC REPORT: AdamW

==========

1. Training Accuracy (Final): 95.78%
2. Cross-Validation: Mean: 0.9564; Std Dev: 0.0015
3. Test Accuracy: 89.67%
4. Final Losses:
   - Final Training Loss: 0.1302
   - Final Test Loss: 0.3768
5. Convergence Speed: 1 epochs to reach target threshold
6. Total Training Time: 243.14 seconds

==========

==========

FINAL METRIC REPORT: RMSprop

==========

1. Training Accuracy (Final): 95.32%
2. Cross-Validation: Mean: 0.9575; Std Dev: 0.0016
3. Test Accuracy: 89.53%
4. Final Losses:
   - Final Training Loss: 0.1456
   - Final Test Loss: 0.3977

5. Convergence Speed: 1 epochs to reach target threshold
6. Total Training Time: 241.16 seconds
==========

==========
FINAL METRIC REPORT: RMSprop_optuna
==========
1. Training Accuracy (Final): 93.55%
2. Cross-Validation: Mean: 0.9461; Std Dev: 0.0018
3. Test Accuracy: 87.94%
4. Final Losses:
   - Final Training Loss: 0.2103
   - Final Test Loss: 0.4167
5. Convergence Speed: 1 epochs to reach target threshold
6. Total Training Time: 253.69 seconds
==========

6. Results

6.1.1 Summary Table (Configuration View)

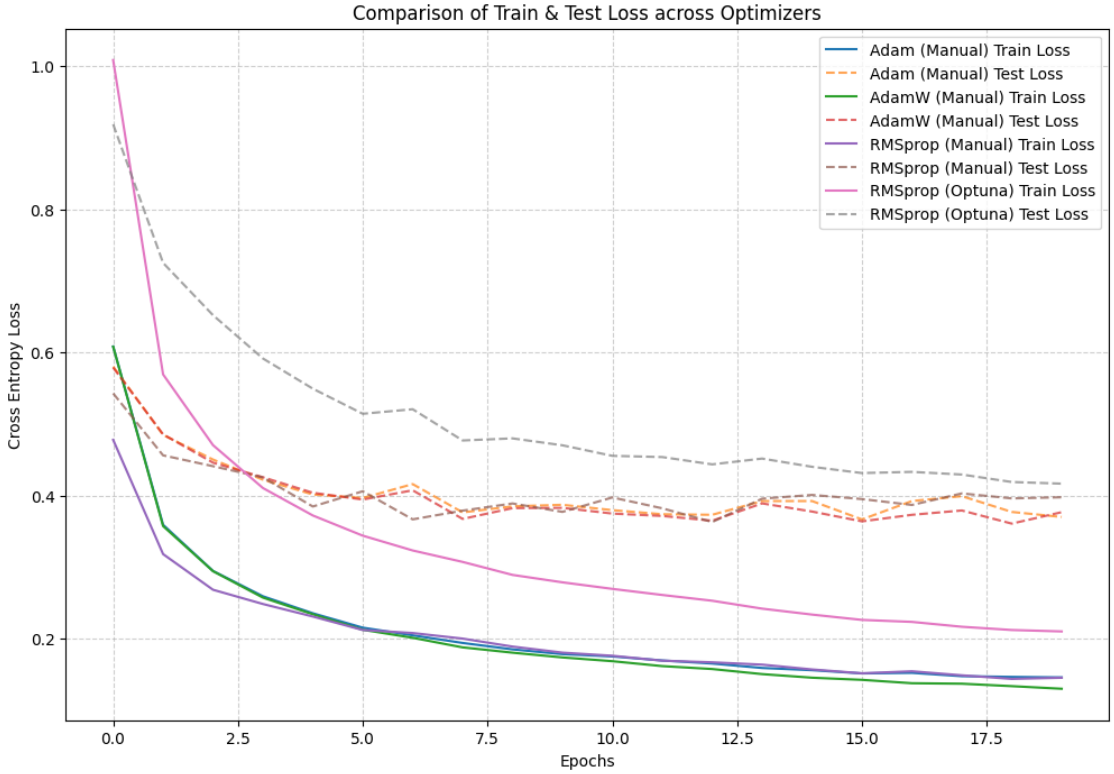
Optimizer	LR	alpha	Eps	Beta_1	Beta_2	momentum	weight_decay	Mean VAL Acc
RMSprop (Baseline)	0.01	0.99	1e-08	n/a	n/a	0.0	0.0	96.06
RMSprop (best)	0.01	0.99	1e-08	n/a	n/a	0.0	0.0	96.06
Adam (Baseline)	0.001	n/a	1e-08	0.9	0.999	n/a	0.0	95.56
Adam (best)	0.001	n/a	1e-08	0.9	default	n/a	0.001	95.60
AdamW (Baseline)	0.001	n/a	1e-08	0.9	0.999	n/a	0.01	95.64
AdamW (best)	0.001	n/a	1e-08	default	default	n/a	0.01	95.64

6.1.2 Summary Table (Metrics View)

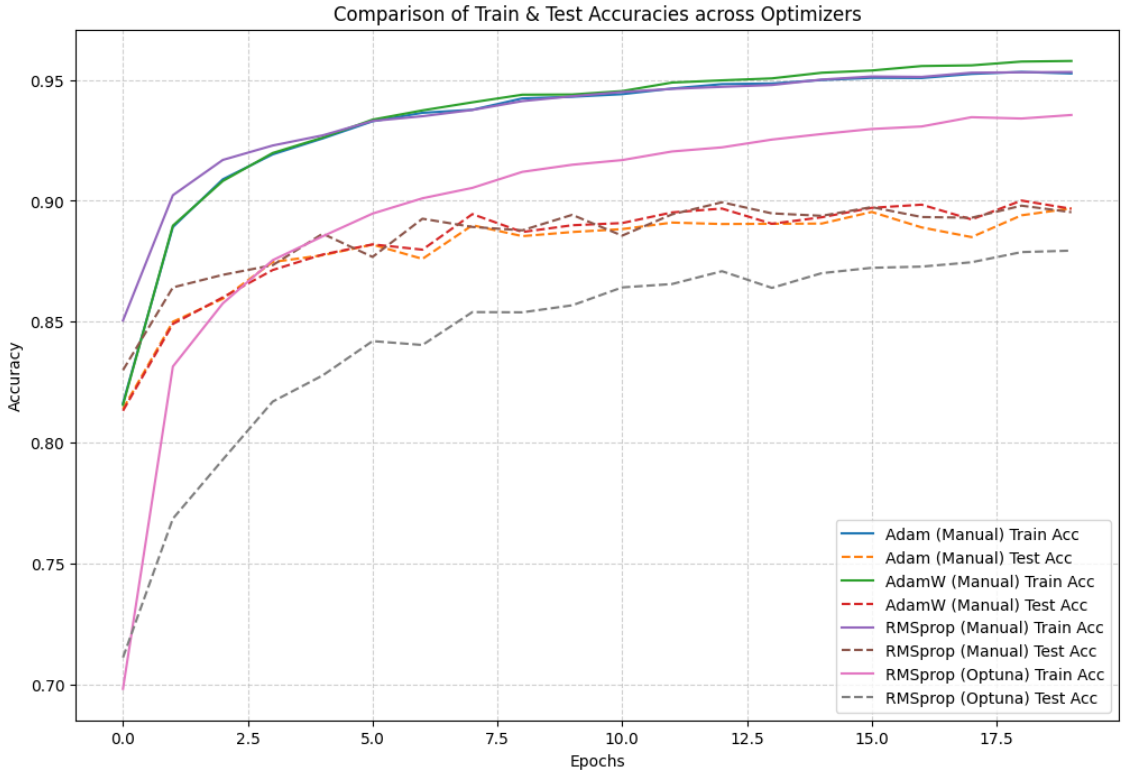
Optimizer	Train Acc	CV Mean	CV STD	Test Acc	Train Loss	Test Loss	Convergence	Time(s)
RMSprop (best)	95.32	0.9575	0.0016	89.53	0.1456	0.3977	1	241
Adam (best)	95.27	0.956	0.0016	89.69	0.1461	0.3702	1	247
AdamW (best)	95.78	0.9564	0.0015	89.67	0.1302	0.3768	1	243

6.2 Required Plots

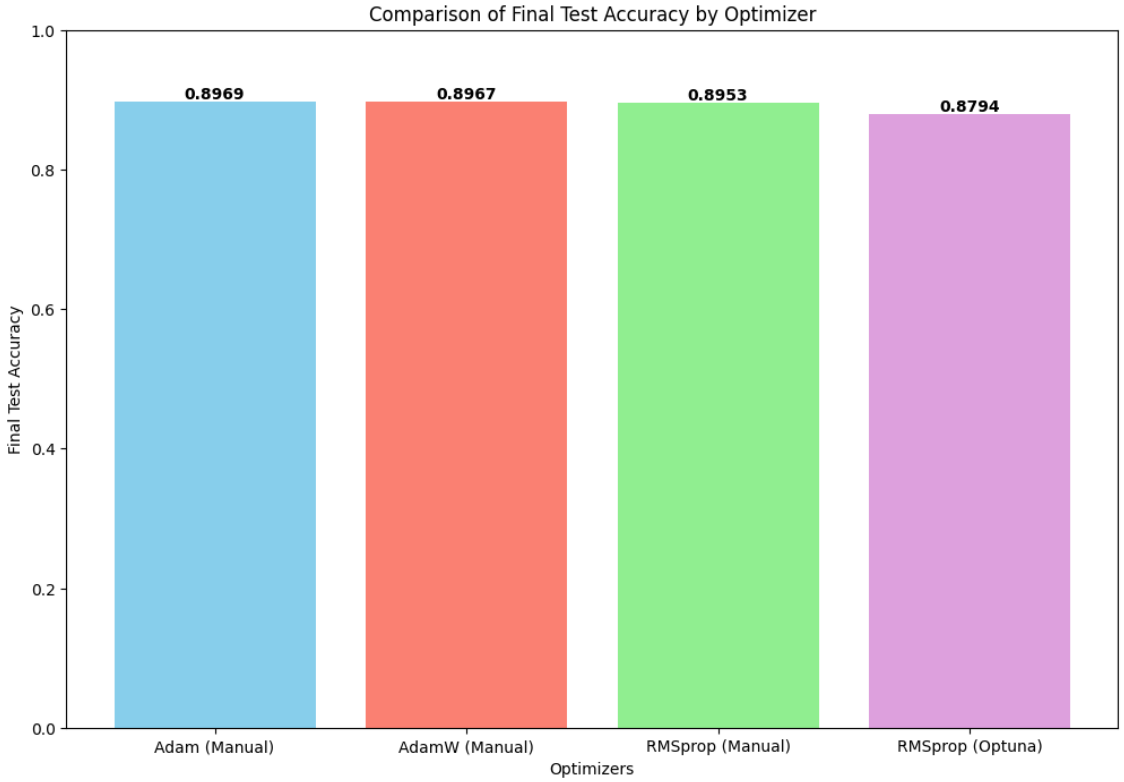
Training & Validation Loss Curves over Epochs



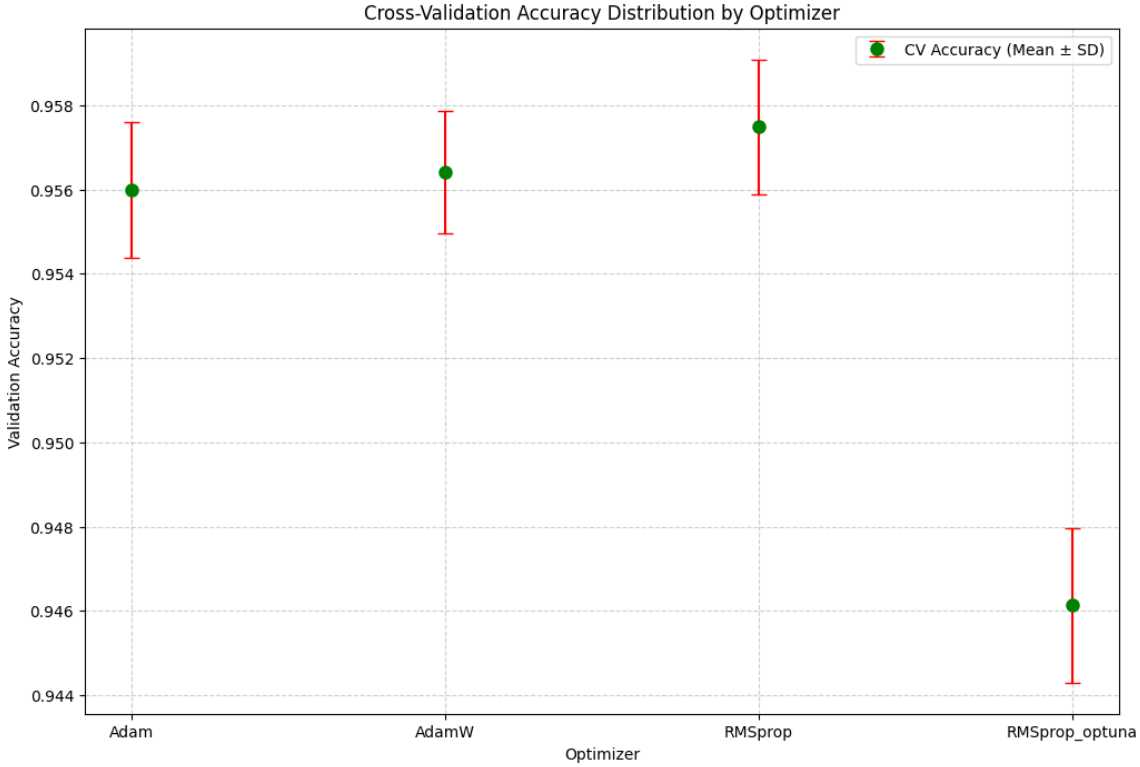
Training & Validation Accuracy Curves over Epochs



Box Chart comparing final test accuracy across optimizers



Error-bar plot for cross-validation accuracy distribution for each optimizer



6.3 Reproducibility

6.3.1: Random Seed

- torch.manual_seed(**498**)

6.3.2 Specific Library Versions

- PyTorch version: 2.12.0
- Torchvision version: 0.27.0
- Scikit-Learn version:1.8.0
- NumPy version: 2.4.6
- Matplotlib version: 3.10.9
- Optuna version: 4.9.0

6.3.3 Confirm Reproducibility

- Interval mini-trials showed the same metrics.

7. Interpretation and Discussion

7.1 Optimizer Analysis

7.1.1 Describe the convergence behavior of each optimizer. Which converged fastest? Which was most stable?

For most configurations across all three optimizers, the training models in the 5-fold cross-validation trials all converged rapidly to achieve the target goal of 90% validation accuracy early (often within the first Epoch). Looking solely at the baseline configurations, RMSprop consistently established a significant early lead and reached validation accuracies between 91.3~92.5% in the first Epoch while Adam and AdamW reached between 90.2~90.9%.

While the convergence speed trend was also observed in the final model training and evaluation with the withheld test set, only AdamW achieved the target test accuracy (90%) at Epoch 19 whereas Adam and RMSprop did not achieve this target by the end of the trial (Epoch 20). This suggests that while RMSprop may have the strongest initial descent (perhaps related to its higher LR at baseline), AdamW may have better stability. This is also supported in the cross-validation trials, revealing that AdamW has a standard deviation of 0.0015 across all trials whereas Adam and RMSprop were evenly matched with a standard deviation of 0.0016.

7.1.2 Discuss how each optimizer responded to learning rate changes. Was any optimizer particularly sensitive or robust?

Adam demonstrated the highest sensitivity to a tenfold adjustment of its LR. The baseline configurations demonstrated consistent training and validation accuracies above 95%, but the LR adjustment (from 0.001 to 0.01) degraded its peak training accuracies to < 90% and validation accuracies to 92~93.

AdamW performed better on the tenfold-increase trial and still achieved training accuracies of around 94% and validation accuracies of 95% on the cross-validation trials. However, its

performance degraded slightly with a tenfold decrease (LR of 0.0001) and dropped to training accuracies of 92% and validation accuracies of 94%.

RMSprop showed the highest resilience and continued to achieve training and validation accuracies of 95% and above with tenfold manipulation of its LR value (from 0.01 to 0.001). These metrics are comparable to the metrics obtained for its baseline configuration.

In summary, our results suggest that Adam is the most sensitive to LR changes while RMSprop is the least sensitive.

7.1.3 Compare the generalization gap (train accuracy vs. test accuracy) across optimizers. What might explain the differences?

One key observation is that amongst all cross-validation trials, the validation accuracies consistently matched or outperformed the training accuracies, so there were no generalization gaps for all three optimizers there.

The final model evaluation using the withheld test set did reveal a non-trivial generalization gap for all three optimizers:

Optimizer	Training Accuracy	Testing Accuracy	Difference
Adam	95.27	89.69	5.58
AdamW	95.78	89.67	6.11
RMSprop	95.32	89.53	5.79

This suggests that even though the data generalized well during the 5-fold cross-validation trials, there were still signs of overfitting when transitioning to the withheld test set. On a global level, it is possible that the withheld test set contained certain noise or additional transformations that were not fully captured in the training data, so the model learned to memorize the training data and was not able to apply these learnings as effectively to the nuances in the new test set.

On an individual level, the generalization gaps between each optimizer were less than 1% apart. While it's surprising to see that AdamW had the largest gap despite its improved version of weight_decay, there may be underlying mechanisms are play.

7.2 Weight Decay and Regularization

7.2.1 Explain, with reference to your results, how decoupled weight decay in AdamW affects generalization compared with Adam.

In theory, based on the findings from our initial research, AdamW is supposed to generalize better compared to Adam. Based on the findings from our cross-validation trials, the baseline values for AdamW showed minor signs of overfitting (0.2% generalization gap) but other configurations showed healthy generalization since the training accuracy was 2% lower than the validation accuracy.

In comparison, most of the final metrics for the Adam trials revealed that the training accuracy was either larger than the validation accuracy, or around 0.2% smaller. While this demonstrate no signs of overfitting, the reverse generalization gap is significantly smaller than AdamW.

Based on the holistic view of results from the cross-validation trials (and not just the final model evaluation alone), it's clear that AdamW has stronger capabilities for generalization.

7.2.2 If AdamW did not outperform Adam in your experiment, provide a hypothesis for why.

Interestingly, the final model training and evaluation showed that AdamW did not outperform Adam. The reason is because while AdamW generalized better across most of its configurations in the HPO trials, the final selection was based on validation accuracy alone. In this manner, the baseline configurations were selected even though they had the worst generalization compared to the configurations. In contrast, the HPO trials for Adam ultimately selected a configuration that included weight_decay (not baseline), which generalized better than its other trials as well. Therefore, when the final model was trained and evaluated on these configurations, Adam ended up outperforming AdamW in terms of generalization. If we ran another trial with one of the other configurations for AdamW, we may likely see a smaller generalization gap.

7.3 Conclusion and Recommendation

7.3.1 State clearly which optimizer performed best on KMNIST and why.

The Adam optimizer performed best based on the final model test accuracy and generalization gap. This was not an overwhelming victory; for instance, the RMSprop optimizer had the highest mean validation accuracy during the HPO trials (95.75% compared to Adam's 95.60% and AdamW's 95.64%) which demonstrated its overall robustness across different configurations. AdamW also demonstrated the highest stability in the cross-validation trials (0.0015 compared to 0.0016) and had the healthiest generalization trend across different configurations (training accuracy < validation accuracy by 2% for most trials).

Ultimately, the Adam optimizer performed best on KMNIST in our trials because the parameters we selected happened to work better for this dataset compared to the parameters selected for AdamW and RMSprop. If we examined the parameter selection process more closely, we could see additional observations:

- **RMSprop**: We tried 18 manually selected configurations + an additional 45 randomly selected configurations through Optuna. Despite the high number of configurations, the best configuration that was selected still underperformed compared to Adam. In this situation, we can claim that Adam likely outperforms RMSprop even though it had a higher mean validation score for the HPO segment.
- **AdamW**: While we expected this optimizer to perform the best, we only tried five manually selected configurations. This ultimately led to the situation where the baseline configuration was selected as the best trial, and its performance fell short of the optimized Adam.

7.3.2 Suggest when you would choose each optimizer in a real-world scenario, beyond this specific task

- **RMSprop**: This optimizer demonstrates more robustness with higher learning rates. For situations where faster convergence is necessary, we would choose the RMSprop for its capability of handling higher learning rates and add early stopping once the target thresholds are reached.
- **AdamW**: If the dataset is a lot more complex like the CIFAR-100, then generalization becomes a key consideration factor. Although the final model and evaluation for AdamW

demonstrated poorer generalization due to the configuration choice, its other configurations in the HPO trials demonstrated a stronger ability to generalize from training to validation/testing. Therefore, AdamW would be a better choice if we take time to conduct hyperparameter tuning.

- **Adam**: Adam is nearly identical to AdamW, so the scenarios to consider are when the decoupled weight_decay feature is not a preferred option. This might be due to strict requirements such as making comparisons against models previously trained on Adam (with the legacy weight_decay). Our research did reveal that Adam now has a new parameter for decoupled weight_decay, so adding this parameter would essentially achieve the same decoupling as AdamW.

7.3.3 Identify at least two limitations of your experimental design and suggest how future work could address them.

- The first limitation is our hyperparameter strategy. After the initial work distribution, team members were allowed to explore and select the hyperparameter configurations at their own discretion if the methodology matched one another. However, this led to an imbalance where we only tried a few configurations for one optimizer (5) and dozens of configurations for another (63), which made it somewhat challenging to compare the results with high confidence. In addition, we also only applied the Optuna approach to one optimizer, and it was challenging to compare the results of that against the other optimizers. In the future, we suggest defining a more strict approach to standardize the number of trials and methodology for exploration (e.g., always check a tenfold increase/decrease while keeping other variables constant).
- The second limitation was the number of Epochs we ran for the trials. The model was able to train very rapidly on the dataset, so it would have been a stronger exploration if we purposefully decreased the learning rate for some of the trials and used that as the baseline before exploring how other parameters contributed to the accuracy/loss over longer Epochs. While the models were able to achieve high final accuracies, it was somewhat challenging to analyze trends that should occur over longer Epochs, like convergence speed and generalization gaps.

Individual Contribution Statements

Sayamon Sittiprom: I was responsible for the systematic investigation of the RMSProp optimizer. I implemented and trained a model on the KMNIST dataset using RMSProp, tuning its hyperparameters through both a manual, systematic search and an automated search using Optuna to identify the best-performing configuration. To compare the resulting configurations, I created visualizations of cross-validation accuracy and train/test performance across optimizers, which showed that manually-tuned RMSProp achieved the strongest overall test accuracy, while the Optuna-tuned configuration required additional training epochs to converge to a comparable result.

Kevin Joseff Cabrera: I was responsible for Optimizer algorithm research and investigation particularly for RMSProp. I looked into the inner workings of RMSProp and subsequently Adam and AdamW and provided base documentation for RMSProp. I worked with Sayamon in running the Optuna Hyperparameter sequence and documenting the results. I provided analytics and insights on the results, mapping relationships between the parameters and uses. I also researched loss landscaping and understanding loss terrain for datasets, mapping the loss landscape generated by the Adam study being the widely-used algorithm among the optimizer list. The loss landscape showed what kind of dataset we are working with and how the optimizers might use it to its advantage.

Thiago Segantini Nogueira: I was responsible for the systematic investigation of the Adam optimizer. I implemented the hyperparameter search strategy, testing four distinct configurations across 5-fold cross-validation on the KMNIST dataset. I investigated the effect of learning rate, momentum ($\beta 1$), and weight decay on convergence speed and final accuracy. I identified a specific failure mode when combining high learning rates with aggressive weight decay, and selected the best stable configuration based on both cross-validation and final training results. I also authored the theoretical background, methodology, and discussion sections for Adam in this report.

Mujtaba Saighani:
I was responsible for researching the AdamW optimizer with weight decay. It is for more complex datasets that need more precise accuracy hence the reason for applying weight decay directly to the weights. AdamW improves on standard Adam by decoupling weight decay from the gradient update. I researched the use of Learning Rate and avoided large generalization gaps in training accuracy and testing accuracy. I ran a hyperparameter search to explore the weight decay parameter together with Learning Rate to find the best configuration for the KMNIST data-set. I found that hyperparameters like Epsilon, Beta1and Beta2 have minimal impact on model performance hence why they were kept at their default values.

Yao-Fu Yang: I was responsible for leading the team for this project, which involved understanding all requirements, breaking down the tasks into equal and parallel responsibilities, and setting up planning resources such as work assignments, tracking, and the GitHub repo. I also built the initial notebook template that included the imports, defining constants, loading the KMNIST dataset, data preparation, defined the standardized model architecture, reusable single-epoch function, reusable k-fold cross-validation function, reusable optimizer trial's function, reusable final model training and evaluate function, metrics reporting, and all final plots and charts as defined from the requirements. I also handled synthesizing the branch codes into the main template, making functions optimizer-agnostic, adding comments and explanations, and completing the final run with our downsized selection of parameters. I was also responsible for progress check-in, quality control, and guidance as needed during the branch investigations.

I also completed the Project introduction/motivation, dataset/model description, and the cross-optimizer interpretation, discussion, and conclusion sections in the final report. I was also responsible for synthesizing the optimizer descriptions and theoretical background, methodology, results, and references from the individual reports for each optimizer while condensing the information to match the project writeup requirements. Lastly, I was responsible for preparing the presentation slides for the Project introduction/motivation, dataset/model description, next steps, and references and will be presenting these during the demonstration.

References

1. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015)*. https://arxiv.org/abs/1412.6980
2. Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *Proceedings of the 7th International Conference on Learning Representations (ICLR 2019)*. https://arxiv.org/abs/1711.05101
3. Tieleman, T., & Hinton, G. (2012). Lecture 6.5 - RMSProp. *COURSERA: Neural Networks for Machine Learning*. https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf
4. Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., & Ha, D. (2018). Deep learning for classical Japanese literature. *NeurIPS 2018 Workshop on Machine Learning for Creativity and Design*. https://arxiv.org/abs/1812.01718
5. *RMSProp Documentation,* pytorch.org https://docs.pytorch.org/docs/2.12/generated/torch.optim.RMSprop.html
6. *RMSProp Optimizer in Deep Learning*, geeksforgeeks.org, https://www.geeksforgeeks.org/deep-learning/rmsprop-optimizer-in-deep-learning/
7. *Lecture Notes*, Hinton, G. (2012). Lecture 6e, Coursera. https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf
8. *Generating Sequences with Recurrent Neural Networks*, Alex Graves, Department of Computer Science University of Toronto https://arxiv.org/pdf/1308.0850v5
9. *Understanding Loss Surfaces*, apxml.com, https://apxml.com/courses/optimization-techniques-ml/chapter-1-ml-optimization-foundations-review/loss-surface-analysis
10. PyTorch. (n.d.). *AdamW — PyTorch 2.12 documentation*. PyTorch Foundation. Retrieved June 21, 2026, from https://docs.pytorch.org/docs/2.12/generated/torch.optim.AdamW.html

Appendix

This report was condensed to fit the format required by the midterm project instructions. However, there was a lot of supporting evidence that was pulled from external trials done as part of each separate branch. We will include some of those findings here:

RMSprop Additional Data and Results

LR	alpha	eps	momentum	weight_decay	TR acc (%)	VAL acc (%)
0.01	0.99	1e-08	n/a	n/a	95.54	96.06
0.01	0.99	1e-08	0.9	n/a	90.08	93.99
0.01	0.8	1e-08	n/a	n/a	94.26	95.58
0.001	0.99	1e-08	n/a	n/a	95.91	96.00
0.0001	0.99	1e-08	n/a	n/a	92.33	94.45
0.01	0.99	1e-08	n/a	0.0001	90.03	94.21
0.01	0.99	1e-08	n/a	0.001	83.44	89.72
0.01	0.99	1e-08	n/a	0.01	73.16	81.49
0.01	0.99	1e-07	0.003	n/a	95.53	95.87

Table. 1 Training Loss per Hyperparameter Configuration (Manual)

ID	LR	alpha	eps	momentum	weight_decay	TR acc (%)	VAL acc (%)
config_1	0.00195	0.8	1.14E-07	0.3702	0.00842	84.26	89.76
config_2	0.00027	0.8025	4.20E-09	0.3709	0.00029	94.62	95.42
config_3	0.02664	0.8033	9.27E-09	0.0847	0.00136	73.04	81.06
config_4	0.03498	0.9781	4.98E-07	0.0887	0.00122	69.89	79.64
config_5	0.00116	0.8218	2.22E-09	0.743	1.05E-05	94.29	95.53
config_6	0.00077	0.9043	9.41E-08	0.6739	0.00191	89.98	93.73
config_7	0.00023	0.894	4.41E-08	0.0824	0.00043	94.33	95.17
config_8	1.24E-05	0.9789	2.02E-09	0.0908	2.83E-05	83.31	87.69
config_9	0.00026	0.8039	3.99E-08	0.4143	9.60E-05	94.64	95.38
config_10	0.00019	0.8016	1.06E-09	0.1199	0.00055	94.00	95.06
config_11	0.00601	0.86	7.38E-07	0.8712	1.03E-05	85.70	91.56
config_12	3.56E-05	0.8478	4.84E-09	0.5868	9.91E-05	91.44	93.89
config_13	0.00342	0.8412	7.39E-09	0.8623	9.98E-05	84.46	90.53
config_14	0.00081	0.8385	2.99E-09	0.3179	1.15E-05	95.24	95.81
config_15	0.00088	0.8825	1.08E-09	0.6248	1.06E-05	95.17	95.78
config_16	6.24E-05	0.8934	1.06E-09	0.5077	2.74E-05	92.80	94.55

config_17	0.01014	0.9244	1.33E-08	0.2142	2.94E-05	91.06	94.39
config_18	0.00059	0.8699	2.92E-09	0.2378	1.75E-05	95.31	95.63
config_19	0.01037	0.9359	1.77E-08	0.5307	5.48E-05	87.03	92.32
config_20	0.08865	0.8779	1.04E-09	0.7404	5.29E-05	48.26	62.43

Table. 2 Training Loss per Hyperparameter Configuration (Optuna - 20l)

Config ID	Optuna Parameter Configuration
config_1	lr: 0.0019476838464710998, alpha: 0.800025289324904, eps: 1.1392983025644397e-07, momentum: 0.3701973165945234, weight_decay: 0.008424534983398027
config_2	lr: 0.0002705494845786357, alpha: 0.8024846649464388, eps: 4.198054585997742e-09, momentum: 0.37094913506955823, weight_decay: 0.0002894154724905874
config_3	lr: 0.026640584134882614, alpha: 0.8032514072130412, eps: 9.270481985035688e-09, momentum: 0.0846827861457791, weight_decay: 0.001360555088824136
config_4	lr: 0.034980420096149156, alpha: 0.9781040889843617, eps: 4.979793789604447e-07, momentum: 0.08872365680660942, weight_decay: 0.0012181438359676958
config_5	lr: 0.0011600597275032583, alpha: 0.8218467767968733, eps: 2.2189459466856255e-09, momentum: 0.742988986089937, weight_decay: 1.0528704406305239e-05
config_6	lr: 0.000768535818087759, alpha: 0.9042889462647967, eps: 9.41157015652493e-08, momentum: 0.673943614017454, weight_decay: 0.0019084424978012544
config_7	lr: 0.0002334045588745601, alpha: 0.8940495196165702, eps: 4.4149719128883496e-08, momentum: 0.08237981675673965, weight_decay: 0.00042734085143992894
config_8	lr: 1.2404761911649825e-05, alpha: 0.9788952427470539, eps: 2.024479373363253e-09, momentum: 0.0907740093921802, weight_decay: 2.829317153761786e-05
config_9	lr: 0.0002550150677454392, alpha: 0.80389512838779, eps: 3.986609347193261e-08, momentum: 0.4143386878657271, weight_decay: 9.601853921513595e-05
config_10	lr: 0.0001930321147327106 5, alpha: 0.8016473102350582, eps: 1.0647457307653659e-09, momentum: 0.1198568520124796, weight_decay: 0.0005523698410613538
config_11	lr: 0.006011183112880236, alpha: 0.8599834287351458, eps: 7.376919810051015e-07, momentum: 0.8711558869622029, weight_decay: 1.0265162904399288e-05
config_12	lr: 3.5621346912319085e-05, alpha: 0.8477740400140249, eps: 4.84430111698863e-09, momentum: 0.5868313213886879, weight_decay: 9.914886573284521e-05
config_13	lr: 0.0034203568468211955, alpha: 0.8411710612200827, eps: 7.387250959134082e-09, momentum: 0.8623258136563945, weight_decay: 9.980396672648531e-05
config_14	lr: 0.0008147881236065796, alpha: 0.8385344255851751, eps: 2.9884607395327503e-09, momentum: 0.31785473211953924, weight_decay: 1.1508825568035366e-05
config_15	lr: 0.0008837377743583721, alpha: 0.8824966424677287, eps: 1.0832281439392694e-09, momentum: 0.6248377417848384, weight_decay: 1.0567445644797476e-05
config_16	lr: 6.238519405532185e-05, alpha: 0.8933857765547827, eps: 1.062935454079886e-09, momentum: 0.5076616613931555, weight_decay: 2.738803779912171e-05
config_17	lr: 0.010139043461371558, alpha: 0.9243817924969993, eps: 1.3303985420504914e-08, momentum: 0.21424910543600087, weight_decay: 2.9396934588483122e-05
config_18	lr: 0.000589660205181521, alpha: 0.8699150195184332, eps: 2.919906946131479e-09, momentum: 0.23781766261182363, weight_decay: 1.7508689909851672e-05
config_19	lr: 0.01037420938580816, alpha: 0.9358530901270731, eps: 1.7719748836962326e-08, momentum: 0.5307023721463947, weight_decay: 5.483216492741157e-05
config_20	lr: 0.08865248939830234, alpha: 0.877904111217427, eps: 1.0372615023712756e-09, momentum: 0.7403977275232295, weight_decay: 5.28516736707329e-05

Table. 3 Config ID Hyperparameter Lookup Table (Optuna - 20)

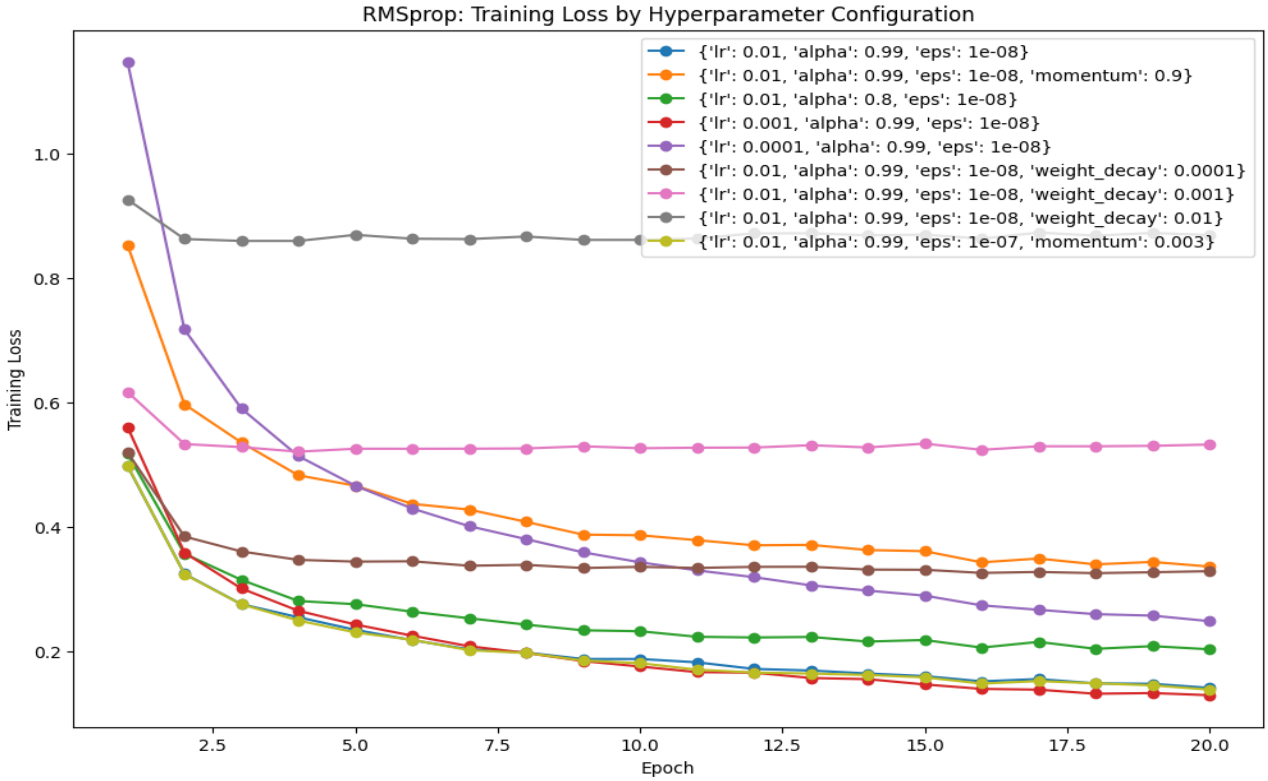


Fig. 1 Training Loss per Hyperparameter Configuration (Manual)

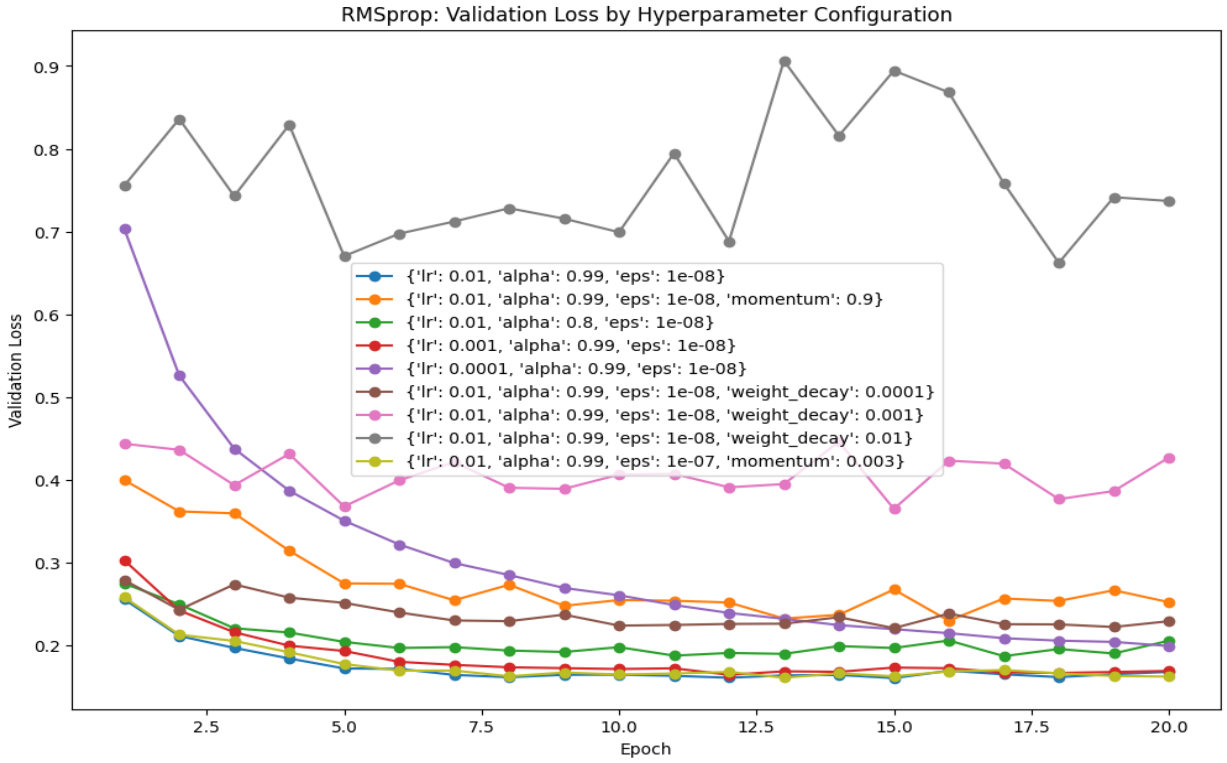


Fig. 2: Validation Loss per Hyperparameter Configuration (Manual)

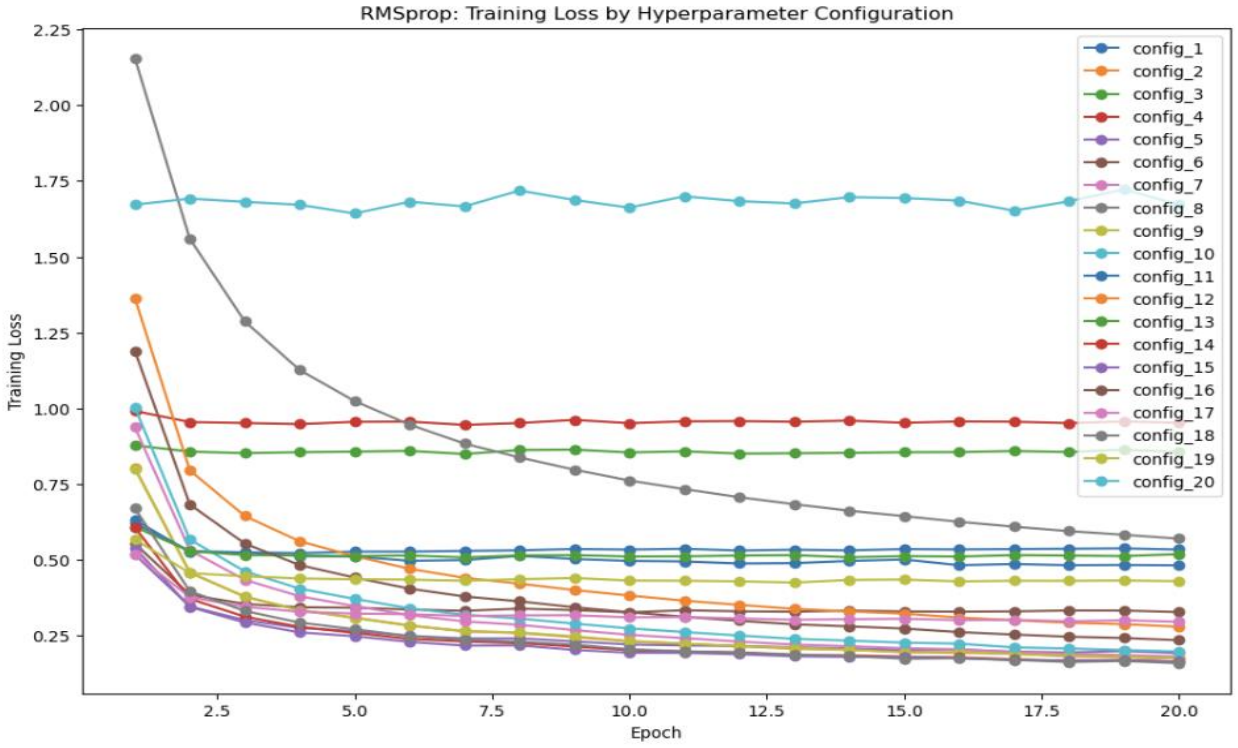


Fig. 3: Training Loss per Hyperparameter Configuration (Optuna - 20)

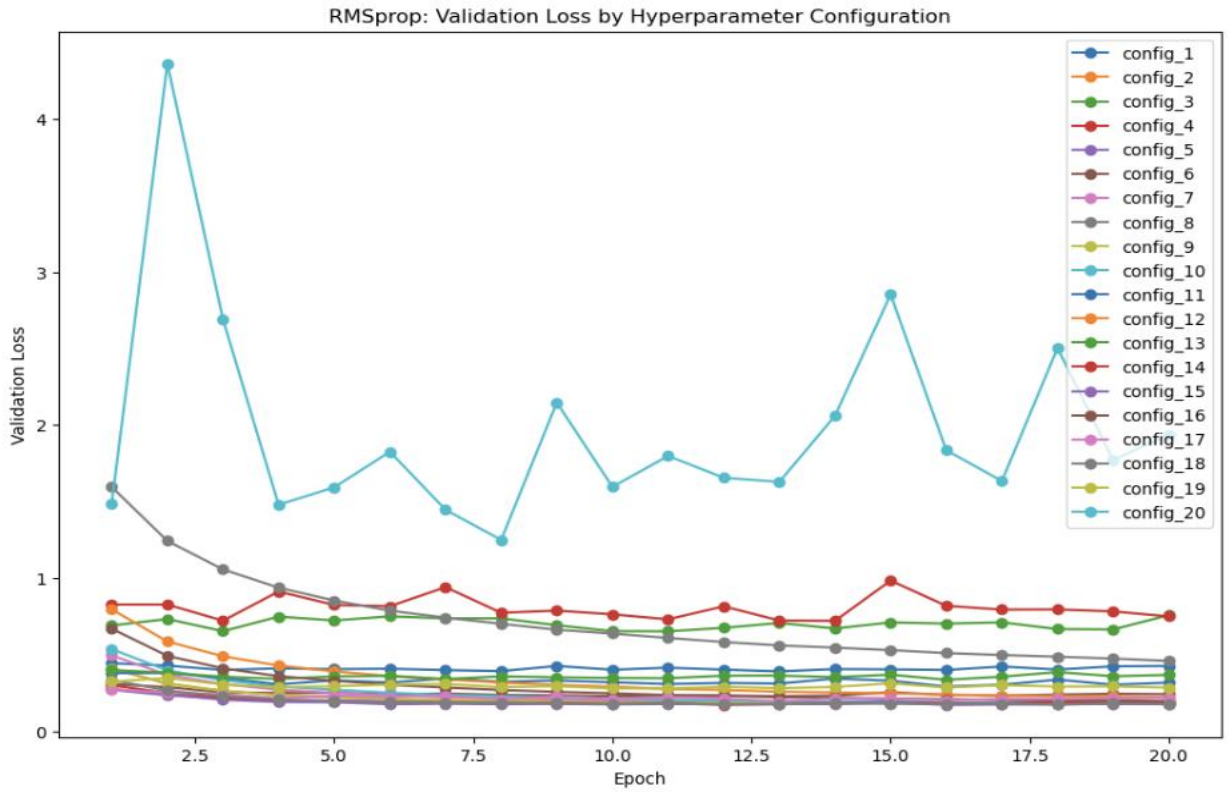


Fig. 4: Validation Loss per Hyperparameter Configuration (Optuna - 20)